

Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 2

Чернов Евгений
НИУ ВШЭ

Git: основы и устройство

Зачем нужна история версий, из чего Git собран внутри и что на самом деле происходит при каждом коммите

План занятия

1. **Зачем Git**
   история проекта вместо папок final_v2
2. **Модель работы**
   снимки, три зоны, первый коммит
3. **Объекты и хеши**
   blob, tree, commit, tag и SHA-1
4. **История и ссылки**
   граф коммитов, ветки, HEAD, папка .git
5. **Отмены и .gitignore**
   restore, amend, reset, что не коммитить
6. **Удалённые репозитории**
   GitHub, clone, push, fetch, pull

Зачем нужна система контроля версий

Вы правите скрипт, а завтра ломается метрика. Что было в проекте вчера в 18:00? Кто поменял эту строку и зачем? Как откатиться, не восстанавливая zip с рабочего стола?

проект без git

```
model.py
model_final.py
model_final_v2.py
model_final_v2_FIX.py
model_final_РАБОЧАЯ.py

# кто, когда и зачем менял —
# неизвестно
```

- **История —** кто, когда и зачем менял каждую строку
- **Откаты —** вернуться к любому сохранённому состоянию
- **Совместная работа —** несколько людей в одном проекте без хаоса
- **Работа без сети —** вся история лежит у вас на диске

01

РАЗДЕЛ 01

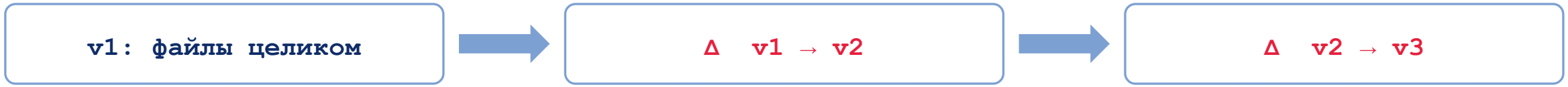
Модель работы

Снимки вместо дельт, три зоны и первый коммит из терминала

Git хранит снимки, а не изменения

Многие системы хранят файл и цепочку правок к нему — дельты. Git в каждом коммите сохраняет снимок всего проекта, а неизменённые файлы не копирует, а ссылается на уже сохранённые.

Другие VCS: первая версия + цепочка дельт



Git: последовательность снимков



Откат — это «встать на другой снимок». Сжатие дельтами внутри pack-файлов — лишь экономия места на диске, не модель.

Настройка и git init

Git подписывает каждый коммит именем и почтой — это поля author внутри коммита, а не аккаунт GitHub. Задаём их один раз, затем превращаем папку в репозиторий.

терминал

```
$ git config --global user.name "Ivan Petrov"
$ git config --global user.email "ivan@edu.hse.ru"
$ git config --global init.defaultBranch main
$ git config --global -l
user.name=Ivan Petrov
user.email=ivan@edu.hse.ru
init.defaultbranch=main
$ mkdir demo && cd demo && git init
Initialized empty Git repository in …/demo/.git/
```

`--global`
файл ~/.gitconfig — для всех ваших репозиториев

`user.name · email`
без них Git откажет или подставит имя@компьютер

`init.defaultBranch`
имя первой ветки; без настройки будет master

`git init`
создаёт .git; ничего никуда не загружает

Три зоны: рабочая копия, индекс, история

У одного файла может быть три версии сразу: на диске, в индексе и в последнем коммите. git status показывает, где они расходятся.

Рабочая копия

`working tree`
файлы на диске, которые вы правите в редакторе

`git add` →

Индекс

`staging · .git/index`
черновик следующего коммита: что и в какой версии войдёт

`git commit` →

История

`.git/objects`
зафиксированные коммиты — неизменяемые снимки

`untracked`
новый файл, Git за ним ещё не следит

`modified`
на диске не так, как в коммите

`staged`
версия лежит в индексе

`committed`
версия сохранена в истории

! **git commit берёт индекс, а не диск:** правка после git add попадёт в коммит, только если снова сделать git add.

Первый цикл: status → add → commit → log

терминал

```
$ echo 'print("hello")' > app.py
$ git status
Untracked files:
        app.py
$ git add app.py
$ git status
Changes to be committed:
        new file:   app.py
$ git commit -m "add app"
[main (root-commit) af9f6b2] add app
 1 file changed, 1 insertion(+)
$ git log --oneline
af9f6b2 add app
```

1. **git status** — что изменилось и где
2. **git add** — положить в индекс
3. **git commit -m** — снимок индекса в историю
4. **git log --oneline** — история, новые сверху

af9f6b2 — начало 40-символьного хеша, имя коммита. **root-commit** — у первого коммита нет родителя.

git diff: два разных вопроса

git diff сравнивает рабочую копию с индексом, git diff --staged — индекс с последним коммитом.

до git add: диск ≠ индекс

```
$ echo 'print("b")' >> app.py
$ git diff
--- a/app.py
+++ b/app.py
@@ -1 +1,2 @@
 print("hello")
+print("b")
```

после git add: индекс ≠ HEAD

```
$ git add app.py
$ git diff
# пусто: диск = индекс
$ git diff --staged
@@ -1 +1,2 @@
 print("hello")
+print("b")
```

`git diff -- app.py`
только один файл

`git show`
последний коммит: сообщение и патч

`git diff --name-only`
только имена изменённых файлов

02

РАЗДЕЛ 02

Как Git устроен внутри

Хеши, четыре типа объектов, граф коммитов и ветки-указатели

Хеш — адрес содержимого

Git — хранилище «ключ → значение». Ключ — SHA-1 от содержимого: 40 шестнадцатеричных символов. Одинаковые байты дают один адрес и хранятся один раз.

терминал (на Linux: sha1sum вместо shasum)

```
$ echo 'hello' | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a
$ echo 'Hello' | git hash-object --stdin
e965047ad7c57865823c7d992b1d046ea66edf78

# хешируется «тип размер\0» + байты:
$ printf 'blob 6\0hello\n' | shasum
ce013625030ba8dba906f756967f9e9ca394464a  -
```

objects/ce/013625…
2 символа — папка, 38 — имя; внутри сжато zlib

hello ≠ Hello
одна буква — совсем другой адрес

целостность
подменить байты незаметно нельзя

git show af9f6b2
хватает короткого однозначного префикса

Четыре типа объектов

blob

содержимое файла: только байты, без имени и прав

tree

каталог: имена, права и хеши blob и вложенных tree

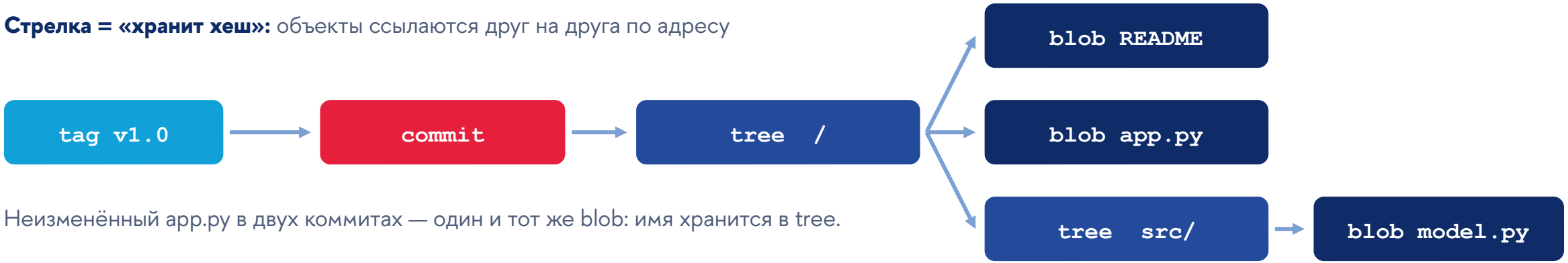
commit

хеш корневого tree, родитель, автор, сообщение

tag

аннотированный тег: имя версии, автор, сообщение

Стрелка = «хранит хеш»: объекты ссылаются друг на друга по адресу



Неизменённый app.py в двух коммитах — один и тот же blob: имя хранится в tree.

Коммит изнутри: git cat-file -p

второй коммит репозитория

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree f60792382ac9f857922a6123df914eaac2ec5200
parent af9f6b2698e679f3876436448922c393b48a525b
author Ivan Petrov <ivan@edu.hse.ru> 1789455720 +0300
committer Ivan Petrov <ivan@edu.hse.ru> 1789455720 +0300

print b
$ git ls-tree --abbrev HEAD
100644 blob 867b3e2    app.py
```

tree
снимок корня проекта на момент коммита

parent
предыдущий коммит; у первого его нет

author · committer
кто и когда: Unix-время и часовой пояс

сообщение
зачем сделано изменение

ls-tree
имя файла хранится в tree, байты — в blob

От HEAD до байтов файла

1. **HEAD** → на какой я ветке
2. **refs/heads/main** → хеш коммита
3. **commit** → хеш корневого tree
4. **tree** → имя и хеш blob
5. **blob** → байты app.py

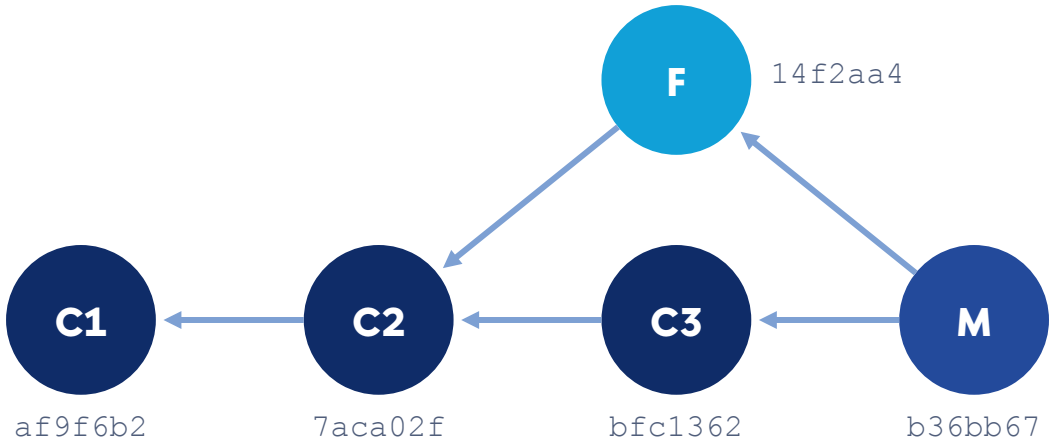
Отдельного «файла в истории» нет — только цепочка ссылок.

первый коммит, шаг за шагом

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
af9f6b2698e679f3876436448922c393b48a525b
$ git cat-file -p af9f6b2
tree ef6ad6746727fa1d0efcf4a020ec3caeda5ccf64
# … author, committer, сообщение
$ git cat-file -p ef6ad67
100644 blob 11b15b1a4584b08fa423a57964bdbf018b0da0d5  app.py
$ git cat-file -p 11b15b1
print("hello")
```

История — направленный граф коммитов

Каждый коммит хранит хеш родителя, поэтому стрелки идут в прошлое. Ветки расходятся, а merge-коммит имеет двух родителей — получается ориентированный ациклический граф (DAG).



стрелка = ссылка на parent; M — merge, у него два родителя

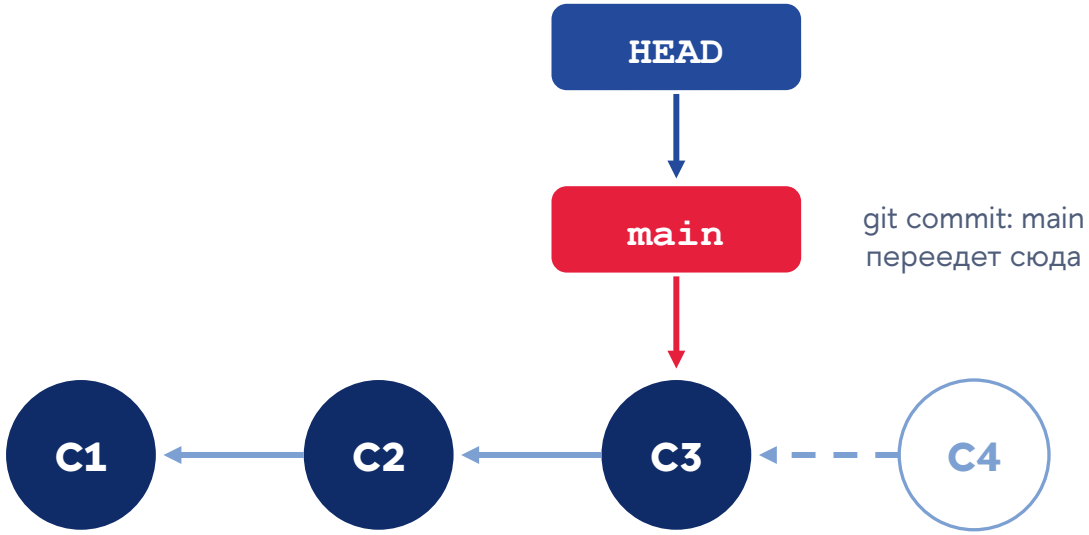
```
git log --oneline --graph

$ git log --oneline --graph
*   b36bb67 Merge branch 'feature'
|\
| * 14f2aa4 add f.py
* | bfc1362 add c3.py
|/
* 7aca02f print b
* af9f6b2 add app
```

Коммит не меняется: его хеш зависит от parent. Поправили старый коммит — новые хеши у него и всех потомков.

Ветка — это указатель на коммит

Ветка — файл в .git/refs/heads с хешем одного коммита: 40 символов и перевод строки, 41 байт. HEAD говорит, на какой ветке вы сейчас. Поэтому ветки в Git почти ничего не стоят.



текущая ветка изнутри

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
bfc13622aa5bf745e19044af42786797e045e97b
$ wc -c < .git/refs/heads/main
41
$ git branch
* main
```

Что лежит внутри .git/

после первых коммитов

```
.git/
├── HEAD
├── config
├── description
├── hooks/
├── index
├── info/
├── logs/
├── objects/
│   ├── 11/b15b1a4584…
│   ├── info/   pack/
└── refs/
    ├── heads/main
    └── tags/
```

HEAD
где вы сейчас: ref: refs/heads/main

objects/
все blob, tree, commit, tag — по хешам

refs/heads/ · tags/
ветки и теги — файлы с хешем

index
индекс; появляется после первого git add

config
настройки этого репозитория

logs/
журнал перемещений ссылок — git reflog

03

РАЗДЕЛ 03

Отмены, .gitignore и remote

Как откатить правку, что не класть в историю и как синхронизироваться с GitHub

Отменить можно по-разному

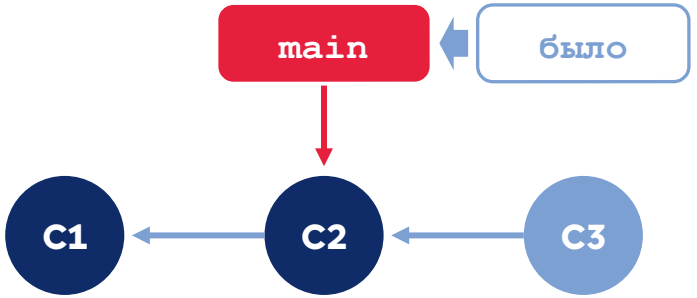
Универсального undo нет: сначала решите, что именно откатываете — файл, индекс или коммит.

Ситуация	Команда	Что произойдёт
испортил файл, ещё не добавлял	`git restore app.py`	файл вернётся к версии из индекса
положил лишнее в индекс	`git restore --staged app.py`	индекс ← HEAD, файл на диске цел
забыл файл в последнем коммите	`git commit --amend --no-edit`	новый коммит с новым хешем
закоммитил то, что не нужно	`git rm --cached notes.txt`	уйдёт из индекса, на диске останется

! **--amend и reset переписывают историю:** так правят только то, что ещё не отправили на сервер (push).

git reset: двигаем ветку назад

git reset HEAD~1 ставит текущую ветку на родителя. Режим решает, что будет с индексом и диском.



терминал

```
$ git reset --soft HEAD~1
# HEAD~1 — родитель, HEAD~2 — на два назад; можно и хеш
```

Режим	Ветка	Индекс	Рабочая копия
`--soft`	назад	не меняется: правки staged	не меняется
`--mixed (по умолч.)`	назад	как в целевом коммите	не меняется: правки на диске
`--hard`	назад	как в целевом коммите	как в целевом: правки пропадут

`git reset HEAD файл` не двигает ветку — это unstage, как git restore --staged.

.gitignore: что не класть в историю

.gitignore — список масок (glob, а не regexp). Такие пути Git не показывает в status и не берёт в add.

.gitignore

```
# секреты и окружение
.env
.venv/
# Python
__pycache__/
*.pyc
# всё в data/, кроме схемы
data/*
!data/schema.sql
# только в корне
/dist
```

Маска	Что значит
`*.pyc`	любое имя на .pyc в любой папке
`build/`	только каталог с таким именем
`/dist`	только в корне репозитория
`**/logs`	logs на любой глубине
`!файл`	исключение из правила выше

Ловушки

закоммиченный файл правило не скроет — `git rm --cached`

из игнорируемой папки файл не вернуть — поэтому `data/*`

Git — это не GitHub

Git — программа на вашем компьютере. GitHub, GitLab, Bitbucket — сервисы, которые хранят копию репозитория и добавляют права доступа, веб-интерфейс, pull request и CI.

git — инструмент

- работает локально и без интернета
- вся история — в папке .git
- команды одни для любого хостинга

GitHub — хостинг

- хранит копию: remote с именем origin
- права: кто читает, кто пушит
- pull request, issues, CI — дальше в курсе

public

клонировать может любой, пушить — только с правами

private

видят только приглашённые; чужим — Repository not found

доступ для push

HTTPS + токен или SSH-ключ:

```
ssh-keygen -t ed25519
```

.pub — в настройки GitHub

Удалённый репозиторий: push, fetch, pull

Ноутбук
локальный .git · main → C3



origin · GitHub
main → C2, про C3 не знает

терминал

```
$ git remote add origin git@github.com:you/demo.git
$ git push -u origin main
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git fetch
   b36bb67..f0bf5dc  main       -> origin/main
$ git pull
```

clone скопировать весь репозиторий к себе

push отправить коммиты, сдвинуть main там

fetch скачать коммиты в origin/main

pull fetch + влить в текущую ветку

Частые ошибки первого дня

Эти сообщения встречаются чаще всего. Каждое прямо говорит, что не так.

`not a git repository`
Почему: нет .git в этой папке и выше: cd в проект или git init

`Please tell me who you are`
Почему: не заданы user.name и user.email

`no changes added to commit`
Почему: правки есть, но не в индексе: забыли git add

`! [rejected] (fetch first)`
Почему: на сервере есть чужие коммиты: git pull, потом push

`src refspec main does not match any`
Почему: ещё нет коммитов или ветка называется master

`Permission denied (publickey)`
Почему: GitHub не знает ваш SSH-ключ: добавьте .pub

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **Коммит — снимок, а не дельта**
   tree проекта + родитель + автор + сообщение

2. **Адрес = хеш содержимого**
   одинаковые байты — один объект; правка — новый хеш

3. **Три зоны**
   рабочая копия → git add → индекс → git commit → история

4. **Ветка — указатель**
   41 байт с хешем; HEAD — где вы; история — граф

Дальше в курсе — **Git: ветвление и воркфлоу**: ветки, слияние, конфликты и pull request.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •